Equipo 3

MapReduce con Hadoop

Agregación con Aggregate · Datos:
Don Quijote de la Mancha



Tabla de Contenido

01 Introducción y ¿qué es MapReduce?
02 Fases: Map, Shuffle y Reduce
03 Ventajas
04 Historia de MapReduce
05 Casos de uso reales
06 Desafíos y límites
07 Descripción del problema
08 Framework Aggregate
09 Resultados en Docker
10 Bug en Hadoop 3.3.6

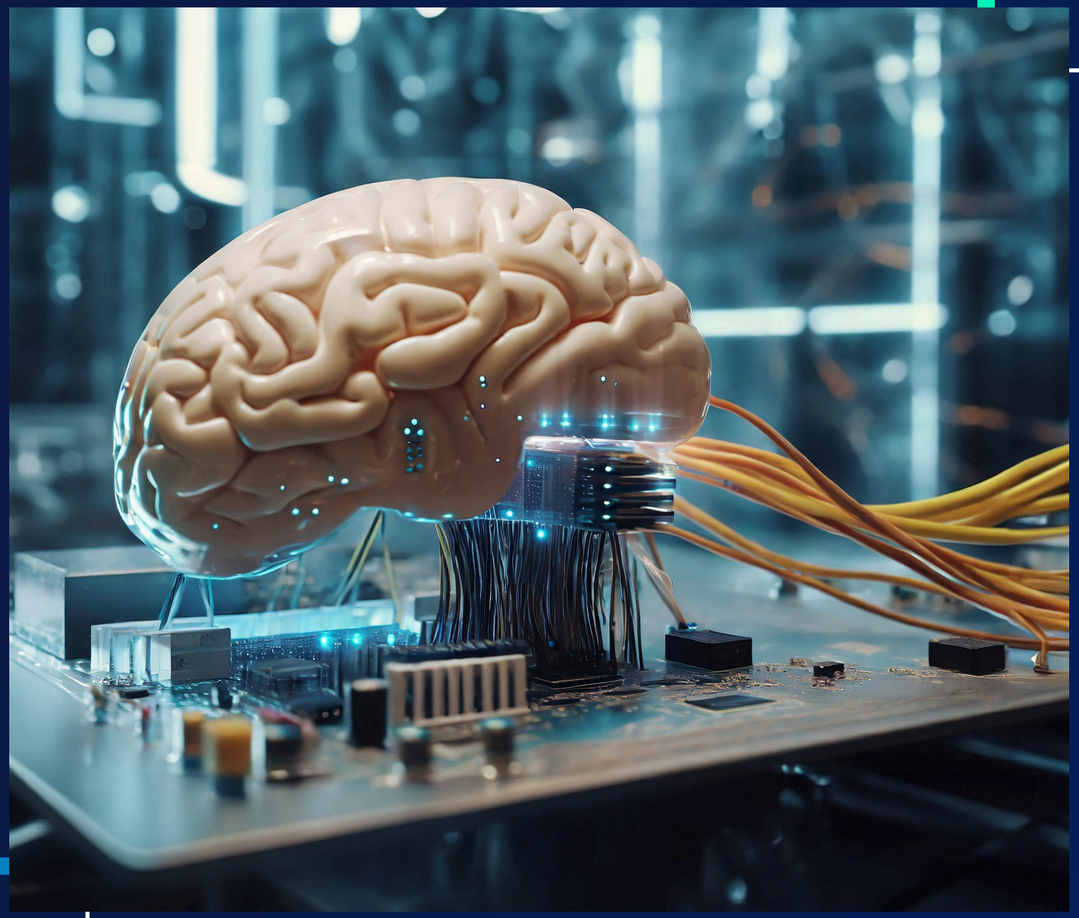


Introducción

Hoy se generan cantidades de datos tan grandes que una sola computadora no puede procesarlas a tiempo. La solución es repartir el trabajo entre muchas máquinas que trabajan en paralelo.

¿Qué es MapReduce?

Es un modelo de programación para procesar grandes volúmenes de datos en paralelo. El programador solo escribe dos funciones, map y reduce; Hadoop se encarga de repartir, agrupar y juntar.



Fases de MapReduce



1. Map

Los datos se dividen en bloques. Cada bloque se procesa por separado y produce pares clave → valor, por ejemplo (palabra, 1).



2. Shuffle

Hadoop junta y ordena automáticamente todos los pares que tienen la misma clave, y los manda al mismo reducer.



3. Reduce

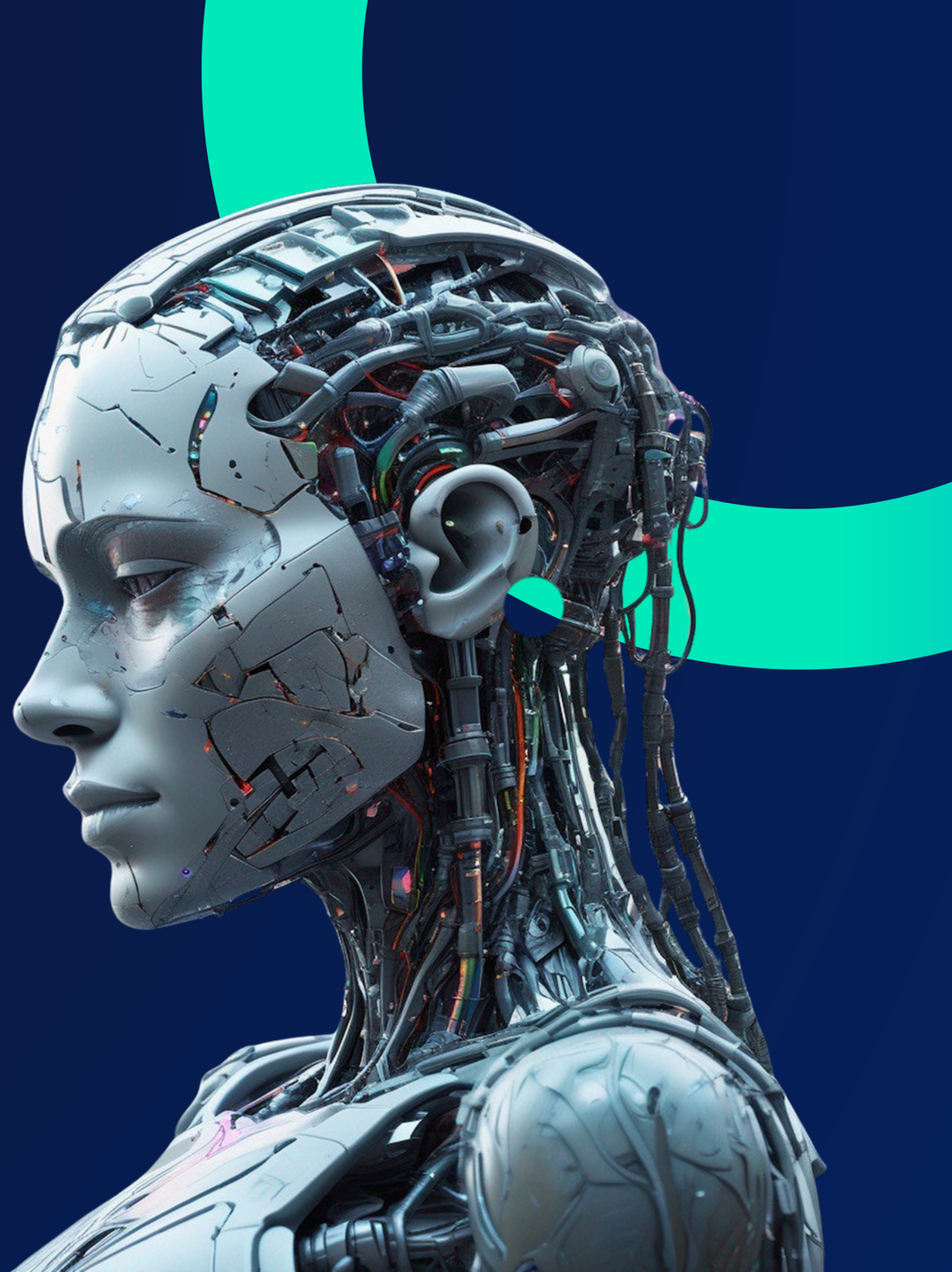
Combina los valores de cada clave (suma, promedio, conteo) y escribe el resultado final en el archivo part-r-00000.

Ventajas de MapReduce

01 Escalabilidad: el mismo programa funciona con una computadora o con miles; solo se agregan más nodos para procesar más datos.

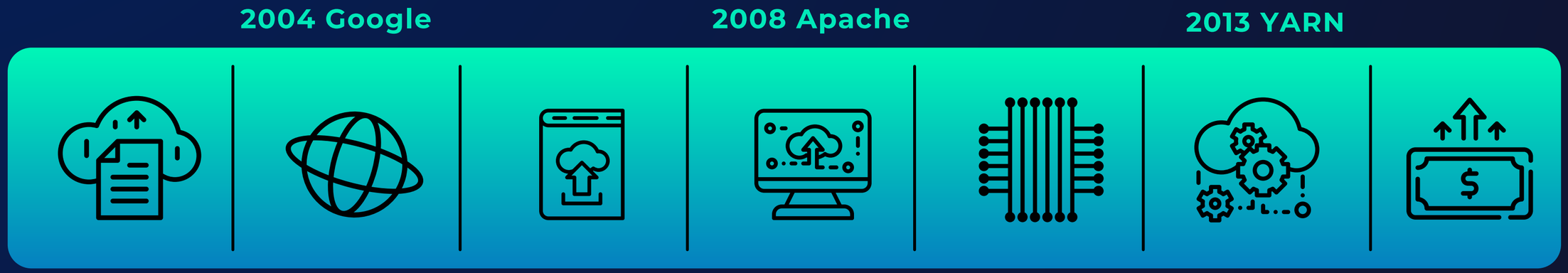
02 Tolerancia a fallos: si una máquina se cae, Hadoop vuelve a ejecutar su tarea en otra, sin perder el trabajo.

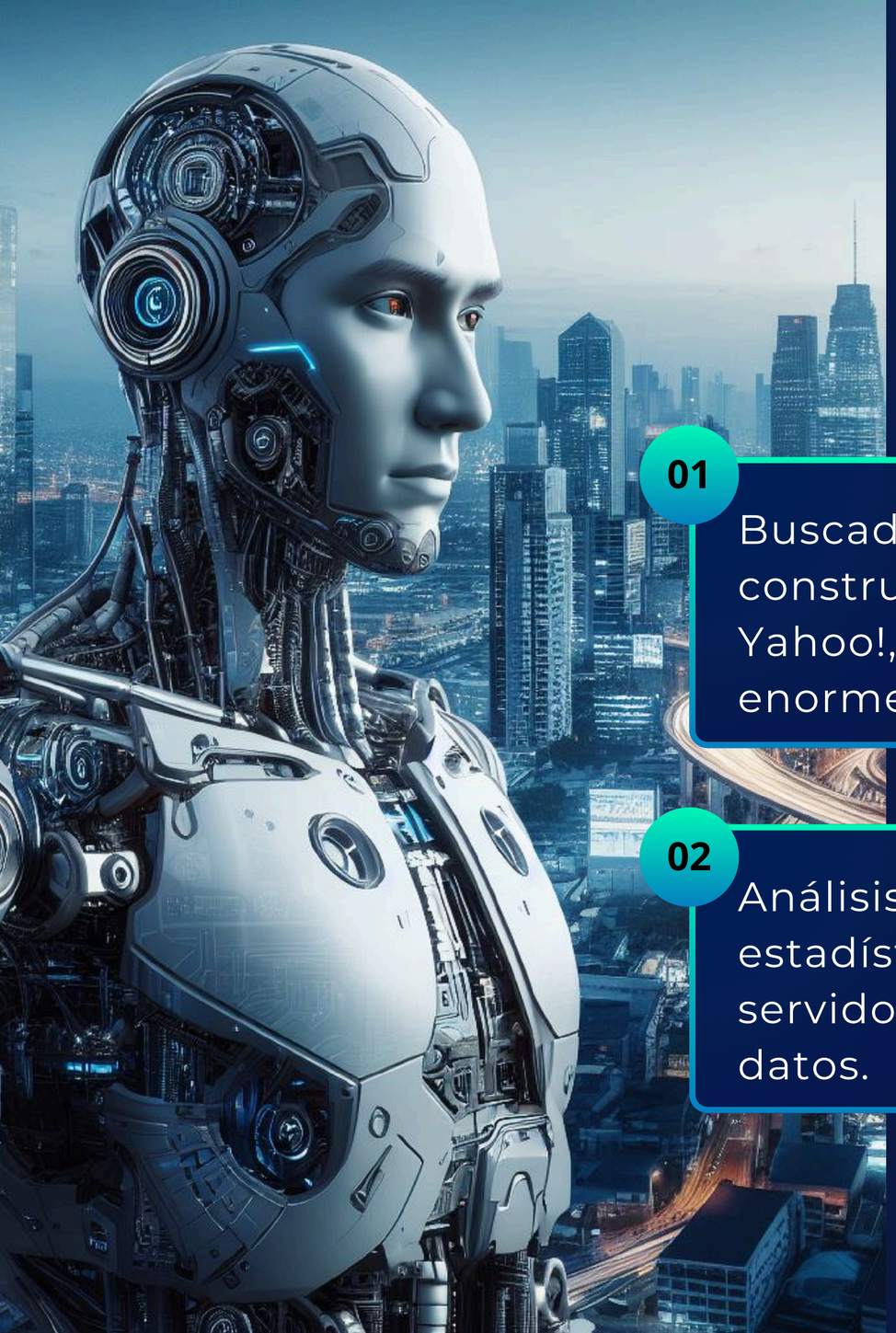
03 Simplicidad: el programador solo escribe map y reduce; repartir, agrupar y juntar los datos lo hace el framework.



Historia de MapReduce

Google publicó cómo procesaba la web con GFS y MapReduce.
Doug Cutting y Mike Cafarella crearon Hadoop como versión libre,
que hoy mantiene la fundación Apache.





Casos de uso reales

01

Buscadores y redes sociales: Google lo usó para construir el índice de la web, y empresas como Yahoo!, Facebook y LinkedIn lo usaron para analizar enormes registros de actividad.

02

Análisis de texto y logs: contar palabras, calcular estadísticas de documentos, revisar registros de servidores o clics, y ordenar grandes conjuntos de datos.

Desafíos y límites

Desafíos de MapReduce

01

Es lento: escribe los resultados intermedios en disco entre la fase Map y la fase Reduce.

02

No sirve para tiempo real: trabaja por lotes, así que la respuesta tarda minutos u horas.

03

Los algoritmos que repiten muchas vueltas (como machine learning) son costosos de programar.

04

Para archivos pequeños no conviene: preparar el trabajo tarda más que procesarlo.

05

Configurar y mantener un clúster de Hadoop requiere conocimientos técnicos.

06

Por eso hoy se combina con herramientas más rápidas como Apache Spark, que trabaja en memoria.

Descripción del problema

Ejemplo: Don Quijote de la Mancha (Project Gutenberg, pg2000.txt)

El problema

Tenemos el texto completo de Don Quijote, un libro con miles de líneas. Queremos saber cuántas veces aparece cada palabra y cómo se distribuyen esas frecuencias. Hacerlo a mano, o leyendo el archivo línea por línea en una sola máquina, es lento y no escala si el texto crece a miles de libros.

La solución con Aggregate

- aggregatewordcount: cuenta cuántas veces aparece cada palabra.
- aggregatewordhist: genera un histograma de esas frecuencias.
- El Map reparte el libro en bloques y etiqueta cada palabra; Hadoop junta y suma los resultados.

Framework Aggregate

Permite hacer MapReduce sin programar el reducer: el map solo etiqueta cada valor con la operación que quiere (ej. LongValueSum:palabra) y Hadoop aplica esa función automáticamente. En Big Data sirve para obtener conteos, sumas, máximos e histogramas de millones de registros de forma rápida.

Sumas y conteos

Máximos y mínimos

Histogramas



aggregatewordcount y aggregatewordhist

Resultados en Docker

Contenedor hadoop · Hadoop 3.3.6 · Entrada: pg2000.txt (Don Quijote)

```
$ ls -l /tmp/equipo3/salida_conteo/
_SUCCESS          0 bytes
part-r-00000     19 bytes

$ head -n 20 /tmp/equipo3/salida_conteo/part-r-00000
record_count    38059

$ head -n 20 /tmp/equipo3/salida_histograma/part-r-00000
record_count    38059
```

38,059

registros contados (record_count)

Los dos trabajos terminaron con _SUCCESS, pero dieron el mismo número: contaron líneas del libro, no palabras.

Bug de Aggregate en Hadoop 3.3.6

¿Por qué los dos programas dieron record_count 38059?

01 Lo que debería hacer

El plugin WordCountPlugInClass lee el texto, separa las palabras y las etiqueta como LongValueSum:palabra para que Hadoop las cuente.

02 El fallo

En el .jar de ejemplos de la versión 3.3.6 hay un error que impide que Hadoop encuentre y cargue ese plugin durante la fase Map.

03 La consecuencia

Sin el plugin, el framework usa su modo por defecto: ignora el contenido y cuenta cada línea como un registro. Por eso solo aparece record_count.

Nuestro Equipo

Equipo 3 · Agregación.
Práctica realizada en un contenedor Docker con Hadoop 3.3.6 usando el texto de Don Quijote de la Mancha.



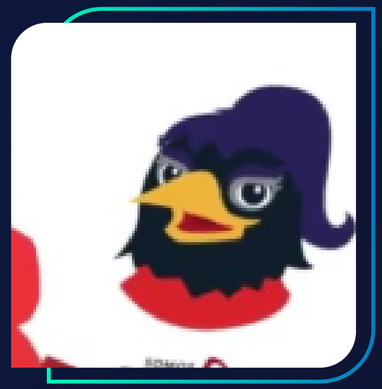
René Nicolas Acosta Lopez

Repositorio GitHub



Germán Eduardo Ruiz Zúñiga

Practica Hadoop



Betel Zurisadai Hernandez Sanchez

Presentación

Muchas Gracias

Página de Recursos

